

```

import os, json, sqlite3, hashlib, logging, re
from datetime import datetime
from apscheduler.schedulers.blocking import BlockingScheduler
import anthropic, urllib.request, urllib.parse

# — CONFIG —————
ANTHROPIC_API_KEY = os.environ['ANTHROPIC_API_KEY']
TELEGRAM_BOT_TOKEN = os.environ['TELEGRAM_BOT_TOKEN']
TELEGRAM_CHAT_ID = os.environ['TELEGRAM_CHAT_ID']
SCAN_INTERVAL_MIN = 15
MIN_IMPACT = ['MEDIUM', 'HIGH']

logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s [%(levelname)s] %(message)s')
log = logging.getLogger('AURUM')
client = anthropic.Anthropic(api_key=ANTHROPIC_API_KEY)

SYSTEM = ('You are AURUM, elite XAUUSD AI. Scan breaking news for gold impact. '
          'Return ONLY valid JSON array. Each item: {id, headline, summary, '
          'category, impact, signal, priceMove, source, timeAgo, reasoning}. '
          'category: trump|fed|geopolitical|inflation|dollar|gold '
          'impact: HIGH|MEDIUM|LOW signal: BULLISH|BEARISH|NEUTRAL')

QUERY = ('Latest breaking news XAUUSD gold price today: Trump tariffs, '
          'Fed rate decision, Middle East conflict, CPI inflation, '
          'USD DXY movement, central bank gold buying')

# — SQLITE DEDUP —————
def init_db():
    conn = sqlite3.connect('sent_alerts.db')
    conn.execute('CREATE TABLE IF NOT EXISTS sent '
                  '(hash TEXT PRIMARY KEY, sent_at TEXT)')
    conn.execute("DELETE FROM sent WHERE sent_at < "
                  "datetime('now', '-6 hours')")
    conn.commit()
    return conn

def is_dupe(conn, headline):
    h = hashlib.md5(headline.encode()).hexdigest()
    return conn.execute('SELECT 1 FROM sent WHERE hash=?', (h,)).fetchone()

def mark_sent(conn, headline):
    h = hashlib.md5(headline.encode()).hexdigest()
    conn.execute('INSERT OR IGNORE INTO sent VALUES (?,?)',

```

```

        (h, datetime.utcnow().isoformat()))
    conn.commit()

# — TELEGRAM —————
def send_telegram(text):
    url = f'https://api.telegram.org/bot{TELEGRAM_BOT_TOKEN}/sendMessage'
    data = urllib.parse.urlencode({
        'chat_id': TELEGRAM_CHAT_ID,
        'text': text, 'parse_mode': 'HTML'
    }).encode()
    urllib.request.urlopen(url, data)

# — FORMAT ALERT —————
def fmt(item):
    imp = item.get('impact','?')
    sig = item.get('signal','?')
    arr = 'UP' if sig=='BULLISH' else ('DOWN' if sig=='BEARISH' else '--')
    cat = item.get('category','').upper()
    return (
        f'[IMPACT: {imp}] [SIGNAL: {sig} {arr}]\n\n'
        f'HEADLINE: {item["headline"]}\n\n'
        f'Signal: {sig} for XAUUSD\n'
        f'Est Move: {item.get("priceMove","N/A")}\n'
        f'Category: {cat}\n\n'
        f'Analysis:\n{item.get("summary","")}\n\n'
        f'Reasoning: {item.get("reasoning","")}\n\n'
        f'Source: {item.get("source","")} | {item.get("timeAgo","")}\n'
        f'—————\n'
        f'AI analysis only – not financial advice.'
    )

# — MAIN SCAN —————
def scan_and_alert():
    log.info('Starting XAUUSD scan...')
    conn = init_db()
    try:
        resp = client.messages.create(
            model='claude-sonnet-4-20250514',
            max_tokens=1500,
            tools=[{'type':'web_search_20250305','name':'web_search'}],
            system=SYSTEM,
            messages=[{'role':'user','content':QUERY}]
        )
        text = ''.join(b.text for b in resp.content if b.type=='text')
        match = re.search(r'\[.*\]', text, re.DOTALL)
        if not match:
            log.warning('No JSON in response'); return

```

```

        items = json.loads(match.group())
        sent = 0
        for item in items:
            if item.get('impact') not in MIN_IMPACT: continue
            if is_dupe(conn, item['headline']):
                log.info(f'Dupe skipped: {item["headline"][:50]}')
                continue
            send_telegram(fmt(item))
            mark_sent(conn, item['headline'])
            sent += 1
            log.info(f'Alert sent: [{item["impact"]} {item["headline"][:60]}')
        log.info(f'Scan done. {sent} alerts sent.')
    except Exception as e:
        log.error(f'Scan error: {e}')
    finally:
        conn.close()

# — ENTRY POINT —————
if __name__ == '__main__':
    import sys
    if '--test' in sys.argv:
        send_telegram('AURUM Agent online. Test alert successful.')
        print('Test alert sent to Telegram.')
    else:
        log.info('AURUM Agent started. Scanning every 15 minutes.')
        scan_and_alert()
        scheduler = BlockingScheduler()
        scheduler.add_job(scan_and_alert, 'interval', minutes=SCAN_INTERVAL_MIN)
        scheduler.start()

```